

The aim of this tutorial is to familiarise you with propositional logic for knowledge-representation and reasoning; that is, how propositional logic can be used to represent knowledge and be used to make inferences. **The ultimate goal is that you are able to use SAT technology to solve problems, by encoding them as a SAT task. This is done in the last part of this tutorial sheet.**

You are not expected to complete all problems in the tutorial session, and you can always catch up and complete unsolved problems later.

A propositional *sentence* or *formula* is:

- A propositional letter, often written as p or q .
- $\neg P$ where P is a sentence.
- $P \wedge Q$ where P and Q are sentences.
- $P \vee Q$ where P and Q are sentences.
- $P \Rightarrow Q$ where P and Q are sentences.
- $P \Leftrightarrow Q$ where P and Q are sentences.

Sometimes we also define *exclusive or* as $P \oplus Q$ where P and Q are sentences.

We refer to $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$ and $\oplus$ as *logical connectives*. In some places, you will find $\rightarrow$ or $\supset$ used instead of $\Rightarrow$ for implication and $\equiv$ instead of $\Leftrightarrow$. In those cases, the symbol $\Leftrightarrow$ is generally used for logical equivalence (i.e., two formulas that have exactly the same truth table).

An *interpretation* in propositional logic is a *truth assignment* (or *valuation*) mapping each atomic propositions to either true or false.

A *truth table* shows the truth value of formulas of interest per possible interpretation of the domain. The truth table showing the semantics (i.e., meaning) of each of the above logical connectives is as follows:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$	$P \oplus Q$
T	T	F	T	T	T	T	F
T	F	F	F	T	F	F	T
F	T	T	F	T	T	F	T
F	F	T	F	F	T	T	F

A propositional formula is:

- **Satisfiable:** if there is at least one truth assignment that makes it true.
- **Valid or Tautology:** if every truth assignment makes it true. A formula that is valid is also satisfiable.
- **Unsatisfiable:** if there is no truth assignment that makes it true (so that all truth assignments make it false).

Two formulas P and Q are *logically equivalent* (denoted $P \equiv Q$) iff $P \Rightarrow Q$ and $Q \Rightarrow P$ (and hence $P \Leftrightarrow Q$) are tautologies. It means their truth value coincide on every interpretation (i.e., they have the same truth table). For example $P \vee Q$ is logically equivalent to $Q \vee P$ and to $\neg(\neg P \wedge \neg Q)$.

PART 1: Basic Logic

- (a) Use truth tables to show that the following are valid (i.e. that the equivalences hold).

$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$	Distribution of $\wedge$
$\neg(P \wedge Q) \Leftrightarrow (\neg P \vee \neg Q)$	de Morgan's Law
$\neg(P \vee Q) \Leftrightarrow (\neg P \wedge \neg Q)$	de Morgan's Law
$(P \Rightarrow Q) \Leftrightarrow (\neg Q \Rightarrow \neg P)$	Contraposition
$(P \Rightarrow Q) \Leftrightarrow (\neg P \vee Q)$	

Solution: These truth tables are very easy to find online.

- (b) For each of the following sentences, decide whether it is
- valid**
- ,
- unsatisfiable**
- , or
- satisfiable but not valid**
- . Firstly, trying “guessing” the answer; then evaluate each properly (e.g. using truth tables). How did your guesses match up?

- i.
- $Smoke \Rightarrow Smoke$

Solution:

Basically, just draw the truth table for each sentence - if the whole column for the sentence is T, then it is valid, if the whole column for the sentence is F, then it is unsatisfiable; neither otherwise. Eg.:

S	$S \Rightarrow S$
T	T
F	T

Therefore, it is valid.

- ii.
- $Smoke \Rightarrow Fire$

Solution:

S	F	$S \Rightarrow F$
T	T	T
T	F	F
F	T	T
F	F	T

Satisfiable but not valid.

- iii.
- $(Smoke \Rightarrow Fire) \Rightarrow (\neg Smoke \Rightarrow \neg Fire)$

Solution: $A : (S \Rightarrow F)$ $B : (\neg S \Rightarrow \neg F)$

S	F	$\neg S$	$\neg F$	A	B	$A \Rightarrow B$
T	T	F	F	T	T	T
T	F	F	T	F	T	T
F	T	T	F	T	F	F
F	F	T	T	T	T	T

Satisfiable but not valid.

- iv.
- $Smoke \vee Fire \vee \neg Fire$

Solution: Your turn!

- v.
- $((Smoke \wedge Heat) \Rightarrow Fire) \Leftrightarrow ((Smoke \Rightarrow Fire) \vee (Heat \Rightarrow Fire))$

Solution: Your turn!

- vi.
- $(Smoke \Rightarrow Fire) \Rightarrow ((Smoke \wedge Heat) \Rightarrow Fire)$

Solution:

S	F	H	$S \Rightarrow F$	$S \wedge H$	$S \wedge H \Rightarrow F$	$(S \Rightarrow F) \Rightarrow (S \wedge H \Rightarrow F)$
T	T	T	T	T	T	T
T	T	F	T	F	T	T
T	F	T	F	T	F	T
T	F	F	F	F	T	T
F	T	T	T	F	T	T
F	T	F	T	F	T	T
F	F	T	T	F	T	T
F	F	F	T	F	T	T

This is a validity (or tautology) as it holds in every possible interpretation.

To see it from another angle, suppose we want to prove that the implication $(S \Rightarrow F) \Rightarrow (S \wedge H \Rightarrow F)$ is true always, that is, it is a validity/tautology:

- Since it is an implication, the only possible way this is true is if $(S \Rightarrow F)$ is true, but $(S \wedge H \Rightarrow F)$ is false.
- Now for $(S \wedge H \Rightarrow F)$ to be false, $S \wedge H$ has to be true and F false.
- But if $S \wedge H$ is true, then S is true.
- Because we already assumed in the first item that $(S \Rightarrow F)$, then together with S being true, we know that F has to be true, which contradicts our second item.
- Hence, the whole implication cannot actually be made false, that is, it is a validity: always true in every possible interpretation.

vii. $Big \vee Dumb \vee (Big \Rightarrow Dumb)$

Solution:

$X : (B \vee D)$

$Y : (B \Rightarrow D)$

B	D	X	Y	$X \vee Y$
T	T	T	T	T
T	F	T	F	T
F	T	T	T	T
F	F	F	T	T

Valid.

viii. $(Big \wedge Dumb) \vee \neg Dumb$

Solution: Your turn!

- (c) Consider the natural language statement “If A then B else C ” (in other words, if A is true then B is true, and otherwise C is true). Give two different formulas in propositional logic that capture that statement formally.

Hints:

The “if-then” conditional statement is represented by logical implication, so “If A then B ” is just $A \Rightarrow B$, have a think about how you could represent “else”

- (d) Prove that $\neg(P \Rightarrow Q)$ and $\neg(\neg Q \Rightarrow \neg P)$ are all equivalent without using a truth-table

Solution: (Note: this is a partial solution)

Of course the truth table would be:

P	Q	$\neg P$	$\neg Q$	$\neg(P \Rightarrow Q)$	$P \wedge \neg Q$	$\neg(\neg Q \Rightarrow \neg P)$
T	T	F	F	F	F	F
T	F	F	T	T	T	T
F	T	T	F	F	F	F
F	F	T	T	F	F	F

Your turn! It is up to you to prove it without the truth table now.

We need to show first that if $\neg(P \Rightarrow Q)$ is true in an interpretation I , then $\neg(\neg Q \Rightarrow \neg P)$ is true in I . Then we need to show that if $\neg(\neg Q \Rightarrow \neg P)$ is true in an interpretation I , then $\neg(P \Rightarrow Q)$ is also true in I .

- (e) Represent the following sentences in propositional logic. Can you prove that the unicorn is mythical? What about magical? Horned?

If the unicorn is mythical, then it is immortal, but if it is not mythical, then it is a mortal mammal.

If the unicorn is either immortal or a mammal, then it is horned. The unicorn is magical if it is horned.

Solution: You can translate the sentences into the following propositional logic expressions:

1. $mythical \Rightarrow \neg mortal$
2. $\neg mythical \Rightarrow mortal \wedge mammal$
3. $\neg mortal \vee mammal \Rightarrow horned$
4. $horned \Rightarrow magical$

From statements (a) and (b), we see that if it is mythical, then it is immortal; otherwise it is a mammal. So it must be either immortal or a mammal, and thus horned. That means it is also magical. However we cannot deduce anything about whether it is mythical.

PART 2: Resolution

First, let us go over an example in full. The resolution method can be used to prove that a CNF formula (a set of clauses) is unsatisfiable, by deriving the empty clause. Consider, for example, the following CNF formula:

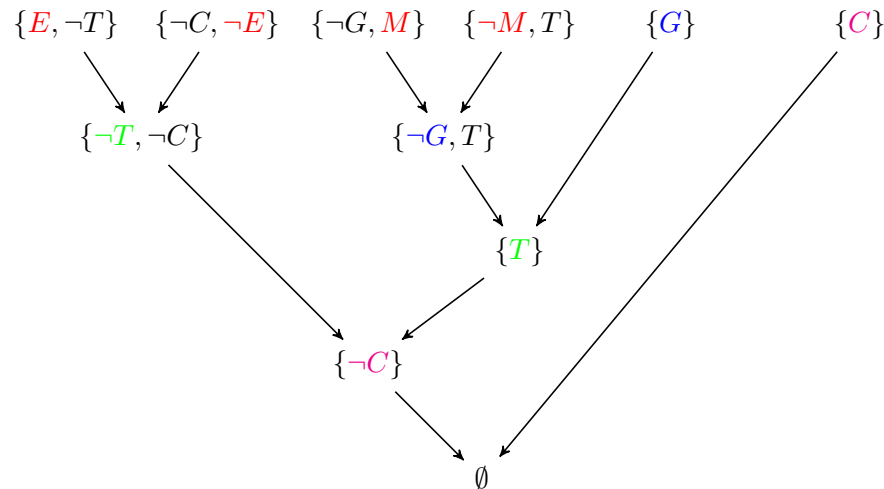
$$\phi = (\neg T \vee E) \wedge (T \vee \neg E) \wedge (M \vee \neg G) \wedge (\neg E \vee \neg C) \wedge (T \vee \neg M) \wedge G \wedge C$$

Here is a resolution proof for the empty clause:

1. $(M \vee \neg G)$ clause in ϕ
2. $(T \vee \neg M)$ clause in ϕ
3. $(\neg G \vee T)$ resolution 1,2
4. $(\neg T \vee E)$ clause in ϕ
5. $(\neg G \vee E)$ resolution 3,4
6. $(\neg E \vee \neg C)$ clause in ϕ
7. $(\neg G \vee \neg C)$ resolution 5,6
8. G clause in ϕ
9. C clause in ϕ
10. $(\neg C)$ resolution 7,8
11. $()$ resolution 9,10

Note that clause $(T \vee \neg E)$ in ϕ was never used, so even if we drop such clause from ϕ , it still remains unsatisfiable. Also observe that, in many cases, there could be more than one (correct) resolution proof deriving the empty clause.

Another way of showing a resolution proof is using a graphical representation of the proof and using sets of literals to denote clauses:



(Colours are added to show the order in which the resolutions occur: red $\Rightarrow$ blue $\Rightarrow$ green $\Rightarrow$ magenta).

OK, now, let's do some exercises....

(a) Use propositional resolution to prove that the following sets of clauses are unsatisfiable. Observe how clauses are now represented via set notation!

- i. $\{\{p, q\}, \{\neg p, r\}, \{\neg p, \neg r\}, \{p, \neg q\}\}$.

Solution:

$$\phi = (p \vee q) \wedge (\neg p \vee r) \wedge (\neg p \vee \neg r) \wedge (p \vee \neg q)$$

Here is a resolution proof for the empty clause (sets of clauses are unsatisfiable):

- | | | |
|----|------------------------|------------------|
| 1. | $(p \vee q)$ | clause in ϕ |
| 2. | $(p \vee \neg q)$ | clause in ϕ |
| 3. | $(p \vee p)$ | resolution 1,2 |
| 4. | $(\neg p \vee r)$ | clause in ϕ |
| 5. | $(\neg p \vee \neg r)$ | clause in ϕ |
| 6. | $(\neg p \vee \neg p)$ | resolution 4,5 |
| 7. | p | resolution 3 |
| 8. | $(\neg p)$ | resolution 6 |
| 9. | $()$ | resolution 7,8 |

It can also solve using a resolution graph.

- ii. $\{\{p, q\}, \{\neg r, s\}, \{\neg p, r, s\}, \{\neg q, \neg r\}, \{p, \neg s\}, \{\neg p, \neg r\}, \{r\}\}$.

Solution: Your turn!

(b) Show that the following statements are true by giving an appropriate resolution proof. You will need to convert formulas to CNF before doing a resolution proof.

- i. $p \wedge (p \Rightarrow q) \models q$

Solution:

Remember that $KB \models \phi$ (i.e., KB entails ϕ : whenever KB is true in an interpretation, so is ϕ) is true **if and only if** $KB \wedge \neg\phi$ is unsatisfiable. To show that $KB \wedge \neg\phi$ is unsatisfiable, we convert KB and $\neg\phi$ into CNF, and do a resolution proof to achieve the empty clause (i.e., a contradiction).

Here, KB is $p \wedge (p \Rightarrow q)$ and query is $\phi = q$. So, we want to show that the following is unsatisfiable:

$$KB \wedge \neg\phi = p \wedge (p \Rightarrow q) \wedge \neg q$$

If we convert this to CNF, we are to prove that the following formula is unsatisfiable:

$$\psi = p \wedge (\neg p \vee q) \wedge \neg q$$

Here is a resolution proof for the empty clause:

- | | | |
|----|-------------------|------------------|
| 1. | $(\neg p \vee q)$ | clause in ψ |
| 2. | p | clause in ψ |
| 3. | q | resolution 1,2 |
| 4. | $(\neg q)$ | clause in ψ |
| 5. | $()$ | resolution 3,4 |

As we have shown that $\psi = KB \wedge \neg\phi$ is unsatisfiable, we know that ϕ must be true and $KB \models q$. ■

- ii. $p \wedge (p \Rightarrow q) \wedge (q \Rightarrow r) \models r$

Solution: (Note: this is a partial solution)

Here, KB is $p \wedge (p \Rightarrow q) \wedge (q \Rightarrow r)$ and query is $\phi = r$. So, we want to show that the following is unsatisfiable:

(Your turn!)

If we convert this to CNF, we are to prove that the following formula is unsatisfiable:

(Your turn!)

Here is a resolution proof for the empty clause:

(Your turn!)

As we have shown that $\psi = KB \wedge \neg\phi$ is unsatisfiable, we know that ϕ must be true and $KB \models r$. ■

- iii. $(p \Rightarrow (q \Rightarrow r)) \wedge (r \Rightarrow s) \wedge p \models \neg q \vee s$

Solution: Your turn!

- iv. $(p \vee r) \wedge (p \Rightarrow (q \vee s)) \wedge (r \Rightarrow t) \models q \vee s \vee t$

Solution:

Here, KB is $(p \vee r) \wedge (p \Rightarrow (q \vee s)) \wedge (r \Rightarrow t)$ and query is $\phi = q \vee s \vee t$. So, we want to show that the following is unsatisfiable:

$$KB \wedge \neg\phi = (p \vee r) \wedge (p \Rightarrow (q \vee s)) \wedge (r \Rightarrow t) \wedge \neg(q \vee s \vee t)$$

If we convert this to CNF, we are to prove that the following formula is unsatisfiable:

$$\psi = (p \vee r) \wedge (\neg p \vee q \vee s) \wedge (\neg r \vee t) \wedge \neg q \wedge \neg s \wedge \neg t$$

Here is a resolution proof for the empty clause:

- | | | |
|----|--------------|------------------|
| 1. | $(p \vee r)$ | clause in ψ |
|----|--------------|------------------|

2. $(\neg p \vee q \vee s)$ clause in ψ
3. $(r \vee q \vee s)$ resolution 1,2
4. $(\neg r \vee t)$ clause in ψ
5. $(q \vee s \vee t)$ resolution 3,4
6. $(\neg q)$ clause in ψ
7. $(s \vee t)$ resolution 5,6
8. $(\neg s)$ clause in ψ
9. t resolution 7,8
10. $(\neg t)$ clause in ψ
11. $()$ resolution 9,10

As we have shown that $\psi = KB \wedge \neg\phi$ is unsatisfiable, we know that ϕ must be true and $KB \models q \vee s \vee t$.

■

v. $p \wedge r \wedge (p \Rightarrow q) \wedge ((q \wedge r) \Rightarrow (s \vee t)) \wedge \neg s \models t$

Solution: Your turn!

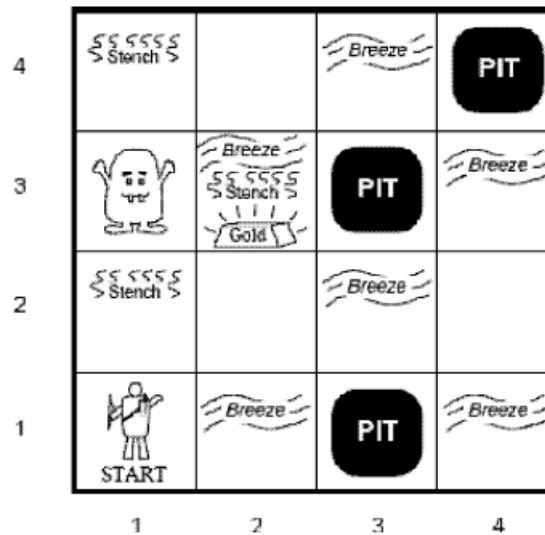
(c) Complete the truth table below. What does this tell you about resolution proofs?

P	Q	R	$\neg P \vee Q$	$P \vee R$	$(\neg P \vee Q) \wedge (P \vee R)$	$Q \vee R$	$((\neg P \vee Q) \wedge (P \vee R)) \Rightarrow (Q \vee R)$
T	T	T	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>
T	T	F	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>
T	F	T	<u>F</u>	<u>T</u>	<u>F</u>	<u>T</u>	<u>T</u>
T	F	F	<u>F</u>	<u>T</u>	<u>F</u>	<u>F</u>	<u>T</u>
F	T	T	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>
F	T	F	<u>T</u>	<u>F</u>	<u>F</u>	<u>T</u>	<u>T</u>
F	F	T	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>
F	F	F	<u>T</u>	<u>F</u>	<u>F</u>	<u>F</u>	<u>T</u>

Solution: As the formula $((\neg P \vee Q) \wedge (P \vee R)) \Rightarrow (Q \vee R)$ is valid, this means that each resolution step is correct, and hence resolution proofs are correct.

PART 3: Knowledge representation.....

(a) For the following Wumpus world:



i. Develop a notation capturing the important propositions.

Solution:

$$S_{xy}, W_{xy}, B_{xy}, G_{xy}, P_{xy}, Y_{xy}$$

The above denotes the sentence “there is a Stench/Wumpus/Breeze/Gold/Pit/You in square [x,y]”.

ii. How would you express in a propositional logic sentence:

α) If square [2, 2] has no smell then the Wumpus is not in this square or any of the adjacent squares?

Solution:

$$\neg S_{22} \Rightarrow \neg W_{22} \wedge \neg W_{21} \wedge \neg W_{12} \wedge \neg W_{32} \wedge \neg W_{23} \quad (1)$$

β) If there is stench in square [1, 2] there must be a Wumpus in this square or any of the adjacent squares?

Solution:

$$S_{12} \Rightarrow W_{11} \vee W_{12} \vee W_{22} \vee W_{13} \quad (2)$$

iii. Given that the agent is standing in square [1,1] and can see (and smell) all adjacent squares, including diagonals; ; how can the agent deduce that the Wumpus is in square [1, 3] using the laws of inference in propositional logic and the formulas constructed in the previous question.

Solution:

We are told that $\neg S_{22} \wedge S_{12}$

Modus Ponens and *And Elimination* applied to the above Equation 1 gives:

$$\neg W_{22} \wedge \neg W_{21} \wedge \neg W_{12} \wedge \neg W_{32} \wedge \neg W_{23}, \quad (3)$$

Modus Ponens applied to Equation 2 gives:

$$W_{11} \vee W_{12} \vee W_{22} \vee W_{13}, \quad (4)$$

Unit Resolution is applied to Equation 4 based on Equation 3, and also we know the agent has been to square [1, 1], hence $\neg W_{11}$. Therefore, W_{13}

- iv. How many propositions are required to fully capture the world in a single formula? What about a general world of size n ?

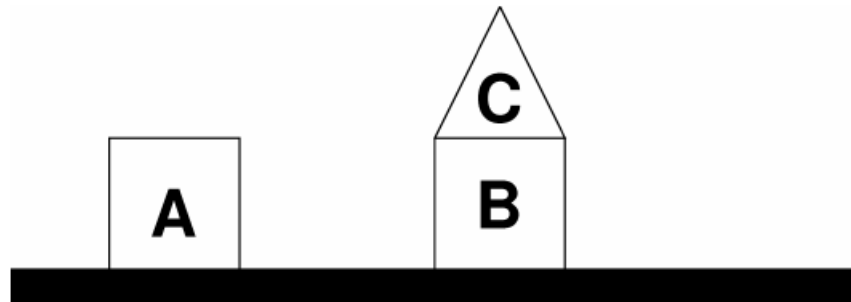
Solution:

For a 4×4 world with 6 propositions, we need $4 \times 4 \times 6 = 96$

$$\neg S_{11} \wedge \neg W_{11} \wedge \neg B_{11} \wedge \neg G_{11} \wedge \neg P_{11} \wedge Y_{11} \wedge S_{12} \wedge \neg W_{12} \wedge \dots \wedge P_{44} \wedge \neg Y_{44}$$

Generally, we would need $6n^2$ propositions.

- (b) Represent the following “blocks world” scene in propositional calculus.



Solution: One example:

$$A_{ontable} \wedge B_{ontable} \wedge C_{onB} \wedge A_{square} \wedge B_{square} \wedge C_{triangle}$$

- (c) Consider the following information available in the domain of thing encountered at sea:

1. None of the unnoticed things are mermaids.
2. Anything entered into the log is sure to be worth remembering.
3. I have never seen anything worth remembering when on a voyage.
4. Anything that is noticed is sure to be recorded in the log.

Formalise the problem in propositional logic and use resolution to formally prove that *I have never seen a mermaid at sea* using the following atomic propositions:

L : entered into the log

M : is a mermaid

S : seen by me

N : noticed

W : worth remembering

Solution:

Assertion	Implication	Clause in KB
1	$\neg N \Rightarrow \neg M$	$(N \vee \neg M)$
2	$L \Rightarrow W$	$(\neg L \vee W)$
3	$S \Rightarrow \neg W$	$(\neg S \vee \neg W)$
4	$N \Rightarrow L$	$(\neg N \vee L)$

“*I have never seen a mermaid at sea*” can be formally written as $(S \Rightarrow \neg M)$ or, in CNF $(\neg S \vee \neg M)$.

Take KB to be the conjunction of all the CNF formulas in the table above. Then, we must prove that $KB \wedge \neg(\neg S \vee \neg M)$ is unsatisfiable (see, the query has been negated). This is equivalent to proving that $KB \wedge S \wedge M$ (we will call this formula ϕ), which is in CNF, is unsatisfiable.

Here is a resolution proof for the empty clause:

- | | | |
|-----|------------------------|------------------|
| 1. | $(\neg S \vee \neg W)$ | clause in ϕ |
| 2. | $(W \vee \neg L)$ | clause in ϕ |
| 3. | $(\neg S \vee \neg L)$ | resolution 1,2 |
| 4. | $(L \vee \neg N)$ | clause in ϕ |
| 5. | $(\neg S \vee \neg N)$ | resolution 3,4 |
| 6. | $(N \vee \neg M)$ | clause in ϕ |
| 7. | $(\neg S \vee \neg M)$ | resolution 5,6 |
| 8. | S | clause in ϕ |
| 9. | $(\neg M)$ | resolution 7,8 |
| 10. | M | clause in ϕ |
| 11. | $()$ | resolution 9,10 |

As we have shown that $KB \wedge S \wedge M$ is unsatisfiable, we know that the query must be true and $(S \Rightarrow \neg M)$. ■

PART 4: Knowledge bases and resolution

- (a) Consider a knowledge base built of just these three weird implications:

$$\begin{aligned}\neg A &\Rightarrow B \\ B &\Rightarrow A \\ A &\Rightarrow (C \wedge D)\end{aligned}$$

- i. Prove formula $A \wedge C \wedge D$ using Modus Ponens only, or explain why this is not possible.

Solution:

It is not possible using Modus Ponens only: Modus Ponens is not applicable to any pair of formulas in the knowledge base. An example of using Modus Ponens is:

When it is cold, I always wear my jacket ($C \Rightarrow J$)

It is cold (C)

Therefore, I am wearing my jacket (J)

- ii. Prove formula $A \wedge C \wedge D$ using resolution.

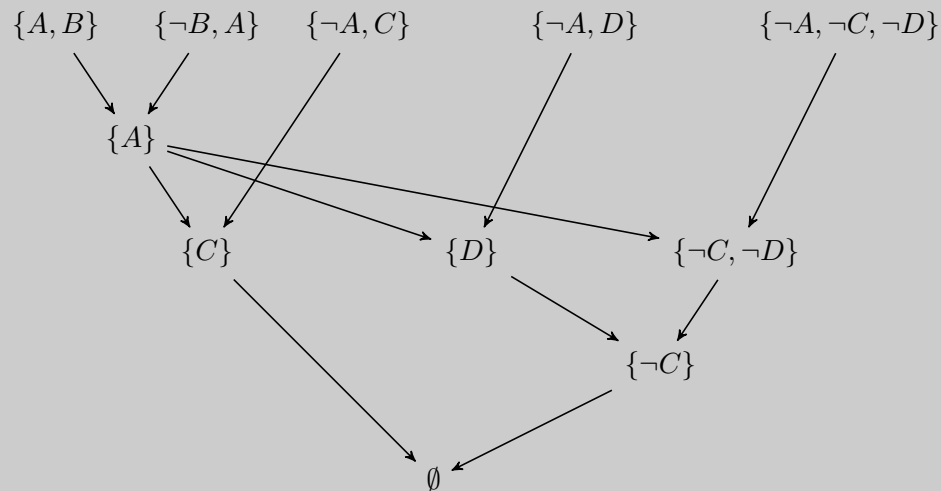
Solution:

Proof by refutation:

1. $A \vee B$ premise.
2. $\neg B \vee A$ premise.
3. $\neg A \vee C$ premise.
4. $\neg A \vee D$ premise.
5. $\neg A \vee \neg C \vee \neg D$ negated thesis.
6. A resolution 1, 2.
7. C resolution 3, 6
8. D resolution 4, 6.

9. $\neg C \vee \neg D$ resolution 5, 6.
 10. $\neg D$ resolution 7, 9.
 11. $\perp$ resolution 8, 10

This can also be proved using a resolution tree:



Here we use the clausal set notation where set $\{P, \neg Q\}$ denotes clause $(P \vee \neg Q)$

(b) Given the following symbols and sentences:

C to indicate that Gianni is a climber;

F to indicate that Gianni is fit; L to indicate that Gianni is lucky;

E to indicate that Gianni climbs mount Everest.

i. Formalize the above sentences in propositional logic:

If Gianni is a climber and he is fit, he climbs mount Everest.

If Gianni is not lucky and he is not fit, he does not climb mount Everest.

Gianni is fit.

Solution:

$$(C \wedge F) \Rightarrow E$$

$$(\neg L \wedge \neg F) \Rightarrow \neg E$$

$$F$$

ii. Tell if the KB built in above is consistent, and tell if some of the following sets are models for the above sentences:

$\{\}; \{C, L\}; \{L, E\}; \{F, C, E\}; \{L, F, E\}.$

(Recall that for the binary variables A , B and C ; the set $S = \{A, C\}$ means A and C are true, and B is false. S is said to be a model of KB iff all statements in KB are true for that given assignment of the variables)

Solution: (Note: this is a partial solution)

The KB is consistent: it has at least a model, as the following check shows.

- $\{\}$ is not a model (it models 1 and 2 but not 3).
- $\{C, L\}$ Your turn!
- $\{L, E\}$ Your turn!
- $\{F, C, E\}$ is a model.
- $\{L, F, E\}$ Your turn!

(c) Prove the propositional formula $[(A \Rightarrow C) \vee (B \Rightarrow C)] \Rightarrow [(A \wedge B) \Rightarrow C]$ is valid using resolution.

Solution: (Note: this is a partial solution)

We can prove this with resolution in the following way:

In the over-all implication, we can take the antecedent to be $(A \Rightarrow C) \vee (B \Rightarrow C)$ and the consequent to be $(A \wedge B) \Rightarrow C$.

Putting the antecedent into CNF gives the single clause $(\neg A \vee C \vee \neg B)$.

Putting the consequent into CNF gives $\neg(A \wedge B) \vee C \equiv \neg A \vee \neg B \vee C$.

As we are trying to prove the consequent, it must be negated: $A \wedge B \wedge \neg C$

The formula can now be shown to be valid using resolution:

Your turn!

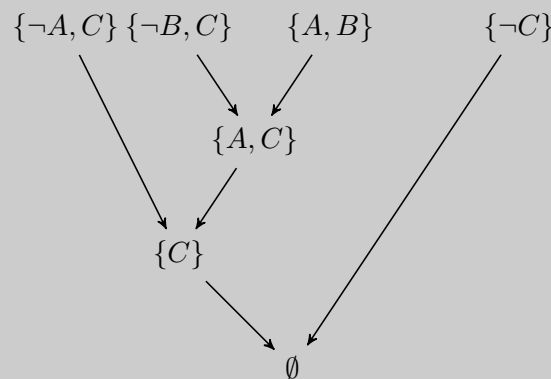
- (d) Let A, B, C be propositional symbols. Given $KB = \{A \Rightarrow C, B \Rightarrow C, A \vee B\}$, tell whether C can be derived from KB or not. Use resolution.

Solution:

C can be derived with Resolution:

1. $\neg A \vee C$.
2. $\neg B \vee C$.
3. $A \vee B$.
4. $\neg C$ negated thesis
5. $B \vee C$ from 1 and 3.
6. C from 2 and 5.
7. $\{\}$ from 4 and 6.

This can also be proved using a resolution tree:



- (e) Heads, I win. Tails, you lose. Use propositional resolution to prove that I always win.

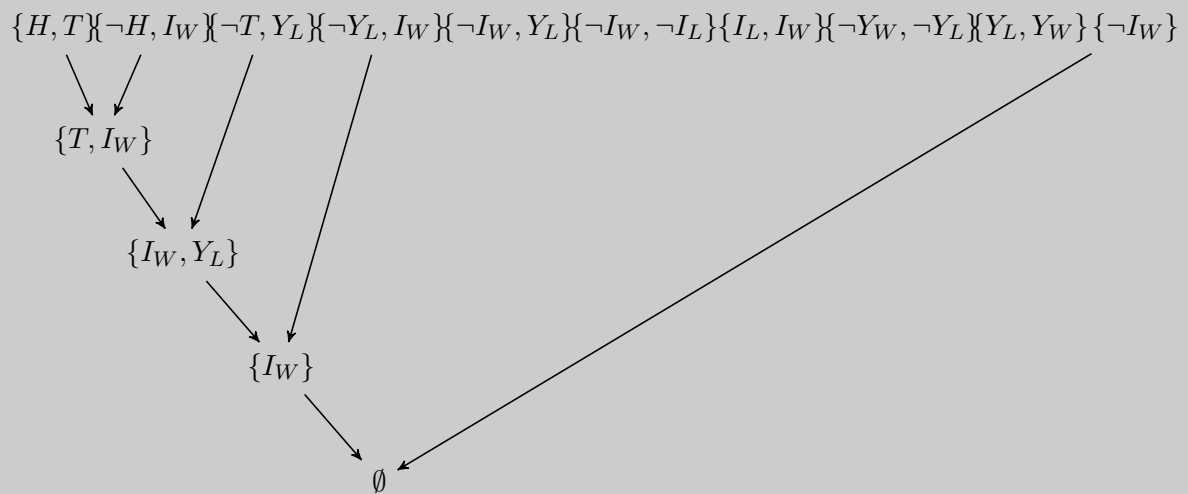
Solution: (Note: this is a partial solution)

We use H and T to signal heads or tails, resp. Also I_W and I_L to denote I win or lose, respectively, and Y_W and Y_L to denote you win or lose, resp. To formalize the problem we have:

1. I win iff I don't lose: $I_W \Leftrightarrow \neg I_L$ or $(I_W \Rightarrow \neg I_L) \wedge (\neg I_L \Rightarrow I_W)$ or $\{\{\neg I_W, \neg I_L\}, \{I_L, I_W\}\}$.
2. You win iff you don't lose:
3. Zero-sum game:
4. Coin either tails or heads:
5. "Heads, I win": $H \Rightarrow I_W \equiv \{\neg H, I_W\}$.

6. “Tails, you lose”:

We need to prove that I always win, that is, that I_W is entailed by the above formulas. We convert all the above to clausal form and then do resolution with those clauses plus $\neg I_W$ and arrive to empty clause.



Here, we can see that there is a lot of redundancy in the KB. Although we have not used all of the clauses, so long as we are able to derive a contradiction, the resolution is valid.

PART 5: SAT Technology

Satisfiability in propositional logic (named SAT) is a very active area of research and used in the industry, for constraint satisfaction, optimisation, planning, etc.

- (a) Research and explain what is the DIMACS format used as a standard for specifying CNF formulas (and used by most SAT solvers). Check, for example [this](#) and [this](#).

Solution:

- The CNF file format is an ASCII file format.
- The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
- The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
- The remainder of the file contains lines defining the clauses, one by one.
- A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
- The definition of a clause may extend beyond a single line of text. The definition of a clause is terminated by a final value of "0".
- The file terminates after the last clause is defined.

- (b) Get a SAT solver (e.g., [MiniSAT SAT](#); [SAT4j](#), file `sat4j.jar`; or Microsoft [Z3 Prover](#)), and test it on some CNF files in DIMACS format from [here](#).

Solution: The official MiniSAT solver can be found [here](#). Binaries are available for Linux and Windows under Cywin.

The [SAT4j](#) solver is Java-based so it runs on any platform, Linux, Mac, or Windows. You just need, of course, Java JRE installed.

What is more, you can even check it with an *online* version of the **Cryptominisat** solver. Just paste the content of your DIMACS file there and click Ready on the upper right corner!

- (c) Finally, solve Part 3.c and ALL the problems in PART 4 by making use of a SAT solver.

Solution: Ultimately, we are to build a DIMACS file encoding the problem as a SAT task.

Next we only the detail the solution for the **sea domain** problem (Part 3.(c)):

```
p cnf 5 6
1 -2 0
-3 4 0
-5 -4 0
-1 3 0
5 0
2 0
```

Now, with this at hand, can you map the DIMACS variables to the actual propositional letters used in the problem?

When ran on SAT4j, you get:

```
[ssardina@Thinkpad-X1 SAT]$ java -jar sat4j.jar MiniSAT sea-domain.cnf
c SAT4J: a SATisfiability library for Java (c) 2004 Daniel Le Berre
c version 1.0.290RC
c org.sat4j.minisat.core.Solver@f6f4d33
c solving tute-07-sea-domain.cnf
c reading problem time 0.012
c #vars      5
c #clauses   4
s UNSATISFIABLE
c Total CPU time (ms) : 0.015
```

When ran in [this online SAT solver](#), the output is (check the word UNSATISFIABLE):

```
c CryptoMiniSat version 5.6.8
c CryptoMiniSat SHA revision 0acd09613073840747161396ac47abf35ec29320
c -- header says num vars:      3
c -- header says num clauses:   5
c -- clauses added: 6
c -- xor clauses added: 0
c -- vars added 5
s UNSATISFIABLE
c ----- FINAL TOTAL SEARCH STATS -----
c restarts           : 0           (0.00      confls per restart)
c blocked restarts   : 0           (0.00      per normal restart)
c decisions           : 0           (0.00      % random)
c propagations        : 0
c decisions/conflicts : 0.00
c conflicts           : 0
c conf lits non-minim : 0           (0.00      lit/conf1)
c conf lits final     : 0.00
c cache hit re-learned cl : 0           (0.00      % of confl)
c red which0          : 0           (0.00      % of confl)
c props/decision      : 0.00
c props/conflict      : 0.00
c 0-depth assigns     : 5           (100.00   % vars)
```

```
c [occ-substr] long subBySub: 0 subByStr: 0 lits-rem-str: 0
c [scc] new: 0 BP 0M T: 0.00
c Conflicts in UIP      : 0
c Mem used              : 0.00 MB
```

Please check this [video](#) that explains this solution in more detail.

End of tutorial worksheet